Date: 27-10-2026

CO-2 - AT-3

APPLICATION BASED QUESTIONS

COURES CODE: DSA0308

COUSRE NAME: NLP

Reg no: 192425034 **Name: Vinay Kandala**

Q2. Error Analysis of Finite-State Morphological Parser in Social Media Data

1. Aim

To identify errors in a finite-state morphological parser that analyzes social media words and modify its transitions to recognize multiple prefixes and suffixes correctly.

2. Problem Description

A company uses a finite-state morphological parser to analyze customer reviews collected from Twitter/X. The parser works correctly for standard English words but fails to identify multiple affixes in words such as happiest, unbelievable, running, reordering, smartphones, and unreadable. The Sentiment140 dataset or another Twitter sentiment dataset can be used for experimentation.

3. Error Identification

Word	Morphological analysis	Possible error
happiest	happy + -est	May fail to handle y-to-i spelling change.
unbelievable	un- + believe + -able	May recognize only one affix.

running	run + -ing	May fail to handle doubled consonant.
reordering	re- + order + -ing	May fail to recognize prefix and suffix together.
smartphones	smart + phone + -s	May fail to recognize compound plus plural suffix.
unreadable	un- + read + -able	May recognize only one affix.

Note: These are expected error types based on the scenario. The assignment does not provide actual parser output, so these are possible failures rather than measured experimental errors.

4. Why the Finite-State Parser Fails

- Single-affix limitation: it may recognize only one prefix or suffix per word.
- Incorrect transition rules: the state machine may not allow a prefix to be followed by a root and then a suffix.
- Spelling variations: happiest and running require spelling-aware morphological rules.
- Compound words: the parser may not recognize multiple lexical components, as in smartphones.
- Incorrect root identification: the parser may identify the wrong root when several affixes are present.

5. Modified Finite-State Transitions

A simplified parser can follow this structure:

START → PREFIX (optional) → ROOT → SUFFIX (optional) → ACCEPT

For words containing multiple affixes, the parser must allow additional prefix and suffix transitions while ensuring that the resulting analysis is valid.

Python Program

```
prefixes = ["un", "re", "dis", "pre"]
suffixes = ["able", "est", "ing", "ed", "s"]


def morphological_parser(word):
    word = word.lower()
    original = word
```

```
    found_prefixes = []
    found_suffixes = []

    # Recognize prefixes
    changed = True
    while changed:
        changed = False
        for prefix in prefixes:
            if word.startswith(prefix) and len(word) > len(prefix) + 2:
                found_prefixes.append(prefix)
                word = word[len(prefix):]
                changed = True
                break

    # Recognize suffixes, longest first
    for suffix in sorted(suffixes, key=len, reverse=True):
        if word.endswith(suffix) and len(word) > len(suffix) + 2:
            found_suffixes.append(suffix)
            word = word[:-len(suffix)]

    # Basic spelling restoration
    if found_suffixes and word.endswith("i") and "est" in found_suffixes:
        word = word[:-1] + "y"

    if "ing" in found_suffixes and len(word) >= 2:
        if word[-1] == word[-2]:
            word = word[:-1]

    return {
        "word": original,
        "prefixes": found_prefixes,
        "root_candidate": word,
        "suffixes": found_suffixes
    }

words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]

for word in words:
    print(morphological_parser(word))
```

Explanation: The program attempts to identify prefixes and suffixes separately and includes simple spelling-restoration rules. It is a simplified rule-based parser, not a complete linguistic finite-state transducer. A production-quality parser would need a lexicon to validate roots and compound-word analysis for smartphones.

6. Evaluation of Parser Accuracy

Select a sample of words from Sentiment140, annotate their correct morphological structures, run both the original and modified parsers, and compare predictions against the annotations.

Accuracy = (Correct predictions / Total predictions) × 100

Actual accuracy values cannot be reported without executing both parsers on an annotated dataset.

7. Computational Complexity

Let n be word length, p the number of prefixes, and s the number of suffixes. The simplified implementation has approximate time complexity O(np + n log s + s). For a small, fixed set of affixes, this is effectively linear in word length. A larger transducer requires more transition rules and potentially a larger lexicon.

8. Result

The parser's limitations were identified, and a simplified parser was developed to recognize multiple prefixes and suffixes. A dataset-based evaluation is required to establish actual improvement in parsing accuracy.

Q4. Morphological Error Analysis in an Information Retrieval System

1. Aim

To analyze stemming errors in an e-commerce product search engine and propose an improved morphological preprocessing strategy.

2. Problem Description

An e-commerce company uses Porter Stemmer to remove suffixes from product-related search terms. Customers search for watches, watching, washable, washer, and washed. Incorrect stemming may combine words with different meanings, reducing the relevance of search results.

3. Error Identification and Classification

Word	Morphological analysis	Category	Potential problem
watches	watch + -es	Inflectional	Incorrect stemming may produce an undesirable search term.
watching	watch + -ing	Inflectional	Suffix removal may help retrieve forms of the same lexeme.
washable	wash + -able	Derivational	May lose the meaning 'capable of being washed'.
washer	wash + -er	Derivational	May confuse a device/person with the verb wash.
washed	wash + -ed	Inflectional	Normalization may help retrieve related verb forms.

Inflectional morphology generally changes grammatical form, whereas derivational morphology can create a new word or change its grammatical category or meaning.

4. Analysis of Stemming Errors

A. Inflectional morphology

Inflectional suffixes such as -s, -es, -ing, and -ed often indicate grammatical properties. Examples include watches → watch, watching → watch, and washed → wash. Combining these forms may improve retrieval when the user intends to search for the same underlying word. Context still matters because watch can refer to a timepiece or the act of observing.

B. Derivational morphology

Derivational suffixes can change a word's meaning or grammatical category. Washable describes something that can be washed, while washer can refer to a device or person that performs washing. Removing derivational suffixes indiscriminately may combine products that are not relevant to the query.

C. Why certain suffixes should not be removed

- Suffixes can create words with different meanings.
- Suffixes can change a verb into a noun or adjective.
- Suffixes may carry useful product-related information.
- Excessive stemming can reduce search precision.

For example, a customer searching for a washer may be looking for a washing-machine component, not information about the verb wash.

5. Improved Morphological Preprocessing Strategy

A better approach combines normalization, lemmatization, preservation of derivational information, and context-aware retrieval.

Proposed workflow: Product descriptions and queries → Text normalization → Tokenization → Morphological analysis → Inflectional normalization → Preserve derivational information → Build search index → Retrieve relevant products.

Python Program

```
import nltk
from nltk.stem import WordNetLemmatizer


nltk.download("wordnet", quiet=True)
nltk.download("omw-1.4", quiet=True)


lemmatizer = WordNetLemmatizer()


words = ["watches", "watching", "washable", "washer", "washed"]


for word in words:
    noun_lemma = lemmatizer.lemmatize(word, pos="n")
    verb_lemma = lemmatizer.lemmatize(word, pos="v")
    print(
```

```
        f"Original: {word:12} "
        f"Noun lemma: {noun_lemma:12} "
        f"Verb lemma: {verb_lemma}"
    )
```

Explanation: The program calculates noun and verb lemmas for comparison. Lemmatization is not a complete solution by itself: output depends on part-of-speech information, and WordNet may not recognize every product-specific use. A production system should use part-of-speech tagging, a domain-specific vocabulary, and relevance testing.

6. Evaluation

Use a subset of product titles and descriptions from the Amazon dataset. Compare the original Porter Stemmer system with the improved system using precision and recall.

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

TP = relevant products retrieved; FP = irrelevant products retrieved; FN = relevant products not retrieved. Actual values must be measured using product queries and relevance labels.

7. Result

The analysis identifies the risks of applying Porter Stemmer indiscriminately to e-commerce search terms. Lemmatization, preservation of derivational information, and context-aware retrieval can help improve search relevance.

Q6. Error Analysis of a Finite-State Morphological Parser

1. Aim

To identify logical errors in a Python program for plural noun identification and correct it to handle regular plurals, words ending in -es, words ending in -ies, and irregular plurals.

2. Given Program

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"


words = ["cars", "boxes", "cities", "children"]
```

```
for w in words:
    print(parser(w))
```

3. Error Identification

Error 1: Incorrect suffix removal

The expression word[:-2] removes the last two characters from every word ending in s. This is incorrect for regular plurals such as cars: cars → ca, while the expected base form is car.

Error 2: Incorrect handling of -es

The program does not distinguish plural patterns. For boxes, the intended base form is box.

Error 3: Incorrect handling of -ies

Words such as cities require a spelling transformation: cities → city.

Error 4: Failure to recognize irregular plurals

Children does not end in s, so the original program labels it singular. The correct analysis is children → child.

Error 5: No morphological validation

The program assumes any word ending in s is plural, although many singular nouns and proper nouns end in s. A rule-based parser needs a lexicon or additional validation.

4. Corrected Python Program

```
def parser(word):
    irregular_plurals = {
        "children": "child",
        "men": "man",
        "women": "woman",
        "mice": "mouse",
        "geese": "goose",
        "teeth": "tooth",
        "feet": "foot",
        "people": "person"
    }

    word = word.lower()
```

```
    # Rule 1: Irregular plurals
    if word in irregular_plurals:
        return irregular_plurals[word], "Plural Noun"

    # Rule 2: Words ending in -ies
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"

    # Rule 3: Common -es plurals
    if word.endswith(("ches", "shes", "sses", "xes", "zes")):
        return word[:-2], "Plural Noun"

    # Rule 4: Regular plurals ending in -s
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"

    return word, "Singular"


words = [
    "cars", "boxes", "cities", "children", "dogs",
    "watches", "men", "books", "cat"
]

for word in words:
    base, category = parser(word)
    print(f"{word:12} -> {base:10} : {category}")
```

5. Expected Output

```
cars         -> car        : Plural Noun
boxes        -> box        : Plural Noun
cities       -> city       : Plural Noun
children     -> child      : Plural Noun
dogs         -> dog        : Plural Noun
watches      -> watch      : Plural Noun
men          -> man        : Plural Noun
books        -> book       : Plural Noun
cat          -> cat        : Singular
```

6. Explanation of Corrections

Rule	Purpose	Example
Irregular plural dictionary	Handles irregular forms	children → child
-ies rule	Restores y	cities → city
-es rule	Removes plural ending	boxes → box
-s rule	Removes regular plural ending	cars → car
Default rule	Retains words without recognized plural form	cat → cat

Rule ordering matters: irregular forms are checked first, followed by more specific suffix patterns, before the general -s rule.

7. Execution Using a Lexical Database

The assignment recommends WordNet or another publicly available English lexicon. The program above uses a manually defined dictionary for irregular plurals and does not integrate WordNet. It is therefore a demonstration of corrected rules, not an experiment with the WordNet database. A larger implementation could use WordNet to validate candidate lemmas.

8. Limitations of Rule-Based Finite-State Parsing

- Lexical ambiguity: a word ending in s is not necessarily plural.
- Irregular forms: a dictionary is needed for many irregular nouns.
- Spelling variations: English plural formation does not follow one suffix rule.
- Unknown vocabulary: new words may not be recognized.
- Context dependence: a word can have different grammatical roles in different sentences.

9. Result

The logical errors in the original parser were identified and corrected. The revised program handles common regular plurals, -es endings, -ies endings, and selected irregular plurals.

Q7. Error Analysis of Morphological Feature Extraction

1. Aim

To identify the preprocessing error in an NLP feature-extraction program and correct the pipeline by applying stemming before vocabulary construction.

2. Given Program

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer


stemmer = PorterStemmer()


documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]


vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)


features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]


print(features)
```

3. Identification of the Error

The primary error is that stemming is performed after feature extraction. The program constructs the vocabulary and document-term matrix with CountVectorizer first, then stems vocabulary words without rebuilding the matrix. The displayed feature list and actual matrix columns no longer correspond.

4. Why This Produces Redundant Vocabulary

The original pipeline is: Documents → CountVectorizer → Original vocabulary and matrix → Stem feature names only → Mismatch between features and matrix. The corrected pipeline stems tokens before constructing the vocabulary and matrix.

5. Corrected Python Program

```
import re
import time
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer


stemmer = PorterStemmer()


documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]


# Original pipeline: no stemming before vectorization
start_original = time.perf_counter()
original_vectorizer = CountVectorizer()
X_original = original_vectorizer.fit_transform(documents)
original_time = time.perf_counter() - start_original
original_vocab = original_vectorizer.get_feature_names_out()


# Corrected pipeline: stem tokens before vectorization
def stem_tokenizer(text):
    tokens = re.findall(r"\b[a-zA-Z]+\b", text.lower())
    return [stemmer.stem(token) for token in tokens]


start_corrected = time.perf_counter()
stemmed_vectorizer = CountVectorizer(
    tokenizer=stem_tokenizer,
    token_pattern=None,
    lowercase=True
)
X_corrected = stemmed_vectorizer.fit_transform(documents)
corrected_time = time.perf_counter() - start_corrected
corrected_vocab = stemmed_vectorizer.get_feature_names_out()


print("Original vocabulary:")
print(list(original_vocab))
print("Original vocabulary size:", len(original_vocab))
print("Original matrix shape:", X_original.shape)
```

```
print("\nCorrected stemmed vocabulary:")
print(list(corrected_vocab))
print("Corrected vocabulary size:", len(corrected_vocab))
print("Corrected matrix shape:", X_corrected.shape)


print("\nOriginal processing time:", original_time)
print("Corrected processing time:", corrected_time)
print("\nCorrected document-term matrix:")
print(X_corrected.toarray())
```

6. Expected Analysis

Porter Stemmer commonly produces the following transformations for the supplied examples:

Original word	**Stem**
running	run
runners	runner
runs	run
studies	studi
studied	studi
studying	studi
organization	organ
organized	organ
organizer	organ

The original sample contains nine distinct word forms. After stemming, the expected vocabulary consists of four stems: run, runner, studi, and organ.

Measure	**Original pipeline**	**Corrected pipeline**
Number of documents	3	3
Vocabulary size	9	4
Document-term matrix columns	9	4

Stemming before vocabulary construction	No	Yes

This reduction is specific to the supplied sample. Results will differ for a larger corpus.

7. Evaluation Using a Real-World Dataset

The assignment requests a real-world corpus such as 20 Newsgroups. The following code compares a baseline without stemming against stemming before feature extraction. An internet connection may be required to download the dataset for the first time.

```
import re
import time
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.pipeline import Pipeline


train_data = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)
test_data = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)


stemmer = PorterStemmer()


def stem_tokenizer(text):
    tokens = re.findall(r"\b[a-zA-Z]+\b", text.lower())
    return [stemmer.stem(token) for token in tokens]


# Baseline: no stemming
baseline = Pipeline([
    ("vectorizer", CountVectorizer(max_features=50000)),
    ("classifier", LogisticRegression(max_iter=1000))
])


start = time.perf_counter()
```

```
baseline.fit(train_data.data, train_data.target)
baseline_time = time.perf_counter() - start
baseline_predictions = baseline.predict(test_data.data)
baseline_accuracy = accuracy_score(
    test_data.target, baseline_predictions
)
baseline_vocab_size = len(
    baseline.named_steps["vectorizer"].vocabulary_
)


# Corrected pipeline: stemming before feature extraction
corrected = Pipeline([
    ("vectorizer", CountVectorizer(
        tokenizer=stem_tokenizer,
        token_pattern=None,
        lowercase=True,
        max_features=50000
    )),
    ("classifier", LogisticRegression(max_iter=1000))
])


start = time.perf_counter()
corrected.fit(train_data.data, train_data.target)
corrected_time = time.perf_counter() - start
corrected_predictions = corrected.predict(test_data.data)
corrected_accuracy = accuracy_score(
    test_data.target, corrected_predictions
)
corrected_vocab_size = len(
    corrected.named_steps["vectorizer"].vocabulary_
)


print("Baseline vocabulary size:", baseline_vocab_size)
print("Stemmed vocabulary size:", corrected_vocab_size)
print("Baseline accuracy:", round(baseline_accuracy, 4))
print("Stemmed accuracy:", round(corrected_accuracy, 4))
print("Baseline training time:", round(baseline_time, 2), "seconds")
print("Stemmed training time:", round(corrected_time, 2), "seconds")
```

8. Interpretation of Evaluation

Vocabulary size indicates the number of distinct features. Stemming may reduce vocabulary by combining word forms. Accuracy measures the proportion of correctly classified test documents.

Stemming may improve accuracy, but it may also reduce accuracy by combining words with different meanings. The measured training time includes vectorization and classifier fitting, but excludes downloading and final test prediction.

9. How the Corrected Pipeline Improves NLP Performance

- Reduced vocabulary redundancy: different word forms may share a stem.
- Consistent feature representation: the feature matrix and vocabulary use the same normalized tokens.
- Potentially lower memory usage: a smaller vocabulary may reduce matrix columns.
- Potentially improved generalization: related word forms can contribute to the same feature.
- Consistent processing: training and testing documents undergo the same transformation.

Stemming is not always beneficial. Its effectiveness depends on the dataset, vocabulary, classification task, and language characteristics.

10. Result

The error in the original program was identified: stemming was applied to feature names after the document-term matrix had already been constructed. The corrected program applies stemming before vocabulary construction. The sample demonstrates a reduction in vocabulary size from nine features to four. The real-world evaluation code compares vocabulary size, classification accuracy, and training time on the 20 Newsgroups dataset.

Conclusion

Q2 and Q4 discuss morphological analysis, finite-state parsing, and preserving meaningful word distinctions. Q6 and Q7 demonstrate how incorrect morphological rules and poorly ordered preprocessing pipelines affect NLP results. The corrected programs offer practical solutions. Where the assignment requires real-world experimentation, the provided code and evaluation procedures should be executed to obtain actual results.

Note: Expected outputs and sample vocabulary counts are based on the supplied examples. Real-world dataset experiments have not been executed in this document; do not present invented accuracy or processing-time measurements as experimental findings.